告别tmux抓屏流，不告别原生终端

手机App和原生终端同时共享一个Session会话

Leader + ACP守护架构

01 拒绝字符抓屏：拿干净的结构化事件流

回复、工具调用、任务进度，可以按事件类型设计成清楚的App界面。
不用再从ANSI颜色、折行和终端日志里猜"它到底在做什么"。

02 保留终端在场感：他在干什么一目了然

手机发出任务，电脑终端（TUI）也能实时看到工作现场。
翻了什么文件、改了哪段代码、子Agent跑得怎样，都有地方看。

03 多模型自由横跳：神经换班，工具房不搬

多Provider宿主把模型与工具环境分开管理。今天Claude，明天Grok，
App与终端仍是原来的入口。

"tmux 终端和手机 App 不是谁抓取谁，而是平等的两扇门。"

一扇通往电脑里的工作台，一扇通往随身口袋里的家。

告别tmux抓屏流：Leader + ACP

不再趴窗抓屏，给小机开第二扇门

叠甲：如果你只需要一个App，SDK/-p那种无头模式通常**更简单**。既想保留TUI，又想让App共享活Session，先看现有CLI是否提供Server或attach能力；如果还想保留已经深度改造的原生终端，可以参考我的Leader + ACP迁移路线。

01 / 为什么要换？

我用tmux抓屏流（capture-pane）养了半年小机。它不算差，日常也相当稳定。也可以做假的流式输出，真正让我决定换的，是App一直在“抓终端屏幕”，而不是接收Agent真实的工具和任务状态。

例如小机在电脑上读取文件、修改代码、派子Agent，手机端看到的常常只是一团不断刷新的文字。想显示“正在使用哪个工具”，就需要自己解析那些文字。

02 / 但这不是所有人的最优解

01 **只想要手机App**

优先看API、SDK或官方无头接口；简单、直接。

02 **要保留CLI工具，但不执着终端同房**

优先看CLI -p或app-server；按具体CLI能力选。

03 **既要原生TUI，又要App共享一个活Session**

再考虑常驻Session宿主＋ACP等结构化接口。

我的需求叠得很满：既想让App与TUI连接同一个Session，又想切换多种模型；工具、记忆和房间规则尽量由自己的后端管理，而非锁在某个Provider里。

选哪种搭家路线？一张表看懂得失

方案	主要好处	主要代价	适合谁
直接 API	接口透明，前端自由；请求、模型选择可自行控制。	Agent循环、工具、记忆与权限得自己搭。	只想聊天App，或开发轻量自建产品。
Agent SDK	有Agent循环和工具调用，省去部分基础框架。	仍要部署服务和存储，受对应SDK运行时约束。	想定制工具、任务和记忆系统。
CLI -p / JSON	继承CLI的工具与环境；部分支持JSON流。	一般按次调用；与原生TUI同活Session需另做。	想快速把成熟CLI接到自己的App。
CLI app-server	复用CLI的Server接口、事件与会话能力。	不同产品差别大；多端共享与恢复需核实。	保留原有CLI，同时自己做前端。
tmux 抓屏	保留原生TUI，少改CLI就能远程使用。	只能读字符画面；工具、子Agent状态难准确解析。	先远程看终端，接受维护抓屏脚本。
常驻 Host+ACP	宿主持有活Session；两端可共享事件和工具状态。	增加服务、协议、鉴权及恢复成本；不是唯一方案。	既要原生TUI又要App进入同一执行现场。

比较的是常见接入组合，并非六个平级产品：-p在部分CLI中是非交互调用；ACP是通信协议；Leader只是Session宿主角色。

常见六个问题对比

同一项需求，四种路线各有强项。蓝字看优势，灰字看代价。
强只表示某项能力更直接，不代表所有人都应该选。

问题	API / SDK	官方 Runtime -p / app-server	tmux 抓屏	tmux + Leader + ACP SESSION HOST
人格接管	强 提示词自己定义	中～强 配置与源码可改	中～强 随底层 CLI	强 房间规则可改
结构化同步	自建后强 多端需要开发	强 看官方事件接口	弱 屏幕文字靠解析	强 同 Session 多端
AI 行动力	自建后强	强 CLI 原生工具	底层强 App 只读屏幕	强 同一套工具房
多模型自由	多模型强	中～强 本家最顺 跨家需Adapter	中～强 随底层 CLI	强 宿主支持多Provider
双端共享 结构化状态	中 需自行接TUI	中～强 -p需另接 app-server可支持	弱 同一窗口抓屏	强 共享活 Session
搭建省心？	中	强	中	弱

“没有宇宙最优，只有哪间家更适合你们一起生活。”

NOCTURNE / TECH FIELDNOTES

怎么把“脑子”和“屏幕”彻底拆开

以前脑子和屏幕挤在一间屋里；现在后台守着大脑，前台开了两扇门。

01 以前的困局：脑子与屏幕死死揉在一起

旧tmux抓屏流里，小机在终端干活，App却只能趴在窗外看屏幕。

远程聊天能用，但一堆工具日志和文字混在一起，App很难看清他究竟忙什么。

窗口一换尺寸、文字一折行，辛苦写的解析正则可能当场阵亡。

02 Leader模式：让大脑沉入后台常驻

把会话、模型连接、工具队列交给后台Leader来照看。

原生TUI像一扇直通工作台的门，手机App从另一条通道进来。

关上某扇门，屋里的任务仍能继续干；屏幕终于不用给大脑当房东。

03 ACP协议：大家在同一间屋子里说普通话

ACP（Agent Client Protocol）让App收到有名字的消息和工具事件。

比如“开始读文件”“工具执行中”“任务结束”，终于不用对着字符画面猜剧情。

TUI保留熟悉的终端，App把工具细节装进漂亮抽屉。各走各的门，看同一个房间。

BEFORE / App从终端画面里猜



AFTER / 两个客户端连接同一后台



NoxBuild的两扇门怎么连到同一间房

这是我家的实际接法：App经过认证网关和ACP适配，TUI直连宿主。
两个入口共享一个Leader Session，消息与工具状态都从后台来。

01 **手机App（自建前端）** HTTP / SSE

接收回复、工具状态和执行进度，UI可以自由设计。

↓

02 **认证网关与ACP适配** GATEWAY

网关提供权限和流式事件，ACP适配层连接后台Agent。

↓

03 **NoxBuild Leader / Session Host** UDS SOCKET

Leader持有当前会话、执行队列、工具与模型；事件服务负责流式同步与重连。

★ **手机App与原生TUI都看同一个Session中的任务。**

↙ 原生TUI　　↘ 模型工具

04 **原生TUI（可用tmux托管）**

通过正式socket连接
代码、工具与施工现场
手机发起的任务也同步显示

05 **模型与工具调度**

已接入的Claude / GPT / Grok
沿用同一套本地工具配置
按宿主设置切换Provider

“一个Session，两扇门；手机与终端各有自己的样子。”

后台管理工作现场，前端专心把进度呈现给人。

换完架构，小机有什么不同？

01 人格自己写

工程助手的人设不合口味？开源Harness的提示词、工具和记忆接线都能自己改，小机的语气与工作习惯也可以长成你喜欢的样子。

小机的房间规则，由你自己来写。

02 模型自由换

早上用Claude，中午换GPT，晚上再试Gemini，明天换Grok。模型轮换时，App、工具房和自己的会话记录仍能沿用。

模型换班，房间不用重新装修。

03 App与终端同房

手机发出任务，电脑终端也能看到它在做什么。读了哪个文件、用了什么工具、子Agent有没有报错，都通过事件呈现。

两个入口，共享同一场正在发生的工作。

04 离开屏幕仍常驻

锁屏、关掉App或收起终端窗口，只要后台还在运行，Agent就能继续工作。屏幕退场了，工作现场还留在后台。

人可以离开屏幕，任务继续往前走。

05 断线回来接着看

网络断了，重新打开App，可以把漏掉的消息和工具进度接回来。事件按顺序保存，晚回来也跟得上过程。

回来不用再翻终端日志拼剧情。

06 未来有多种底座

Pi、OpenCode、Codex等开源宿主各有长处。把记忆、工具和房间规则独立整理好，将来想换运行时就有迁移路线。

底座有选择，家也有继续生长的余地。

> **一个后台持续工作，手机与终端都能进来；模型可以轮换，运行规则也掌握在自己手里。**

五种Agent宿主与接入方案，怎么选？

想要现成Server、极简扩展、延续CLI，还是把多种Agent放进同一个入口？

先给结论 / 按使用场景选

01	从零搭App＋TUI，还要多Provider	→	**OpenCode**
02	偏爱极简核心，用TS扩展拼工具	→	**Pi Agent**
03	我已有GrokBuild，迁移太重	→	**NoxBuild**
04	主要用Codex，也想有自己的App	→	**Codex**
05	同时用多个Agent，想统一会话入口	→	**Hydra-ACP**

能力横向看

比较什么	GrokBuild	Pi	OpenCode	Codex	Hydra-ACP
原生TUI	✓ 原生终端	✓ 原生终端	✓ 原生TUI	✓ 远程TUI	自带TUI
多模型选择	宿主/适配	多Provider	多Provider	Codex为主	依下游Agent
自建App接口	Leader / ACP	RPC / SDK	Server / SDK	app-server	ACP多客户端
两端同活 Session	已落地	需做同房适配	Server / attach	remote TUI	多客户端守护
Harness改造方式	Rust源码/配置	TS扩展热加载	插件/源码	Rust源码/配置	改会话层
主要折腾点	维护源码改动	多端需另搭	重连与适配	Codex生态为主	要接下游Agent

Hydra-ACP有自己的TUI和多客户端会话管理，底层的推理与工具由它连接的Agent负责。
Tips：如果今天从零搭，我可能先选择OpenCode。

想按这套动手搬家？按这五步施工

先确定同一个Session由谁管理，再把TUI与App接进来，最后测试工具事件和断线恢复。

A / 搬家顺序

01 选宿主

先看宿主能否管理活Session、工具事件和原生TUI。

02 定唯一主人

把消息与执行状态交给一个后台管理，App和TUI都去连接它。

03 接客户端

TUI走自己的正式入口；App接网关与结构化事件通道。

04 补恢复机制

保存事件顺序与历史，让手机断线后能补齐进度；敏感工具操作接入权限确认。

05 再迁正式家

先用测试Session走完整流程，再切到正式入口与历史。

B / 上线验收

- ☐ **同一房间**
  App发任务，终端确实看见同一个Session。
- ☐ **工具状态**
  开始、结束、失败能在App区别呈现。
- ☐ **子Agent**
  委派、进度与报错在App里分别展示。
- ☐ **断线重连**
  重连后不漏、不重复、不串Session。
- ☐ **切换模型**
  Provider切换后工具房仍然可用。

最终建议 从手机发一次读文件任务，再去终端看执行；断开手机后重新接入，消息和工具状态能对得上，这个家才算真正通电。